

MEMORIA PRÁCTICA 1
SISTEMAS INFORMÁTICOS:
Microservicios. API REST en ordenador

Alejandro Fernández Fernández

Marcos Jiménez de las Moras

Grupo 1322

Pareja 11

- **SERVICIO DE USUARIOS:**

El microservicio de usuarios gestiona las cuentas del sistema en el puerto 5050 mediante una API REST en formato JSON. Sus tres operaciones principales son el registro de usuarios con el método PUT, el inicio de sesión con POST y la modificación de contraseñas con PATCH.

Por motivos de seguridad, las contraseñas nunca se guardan en texto plano; siempre se procesan con el algoritmo criptográfico SHA-256 antes de almacenarse en memoria. Al registrarse, cada usuario recibe un identificador único aleatorio generado con uuid4 y un token de sesión derivado de dicho identificador y de una clave secreta compartida mediante uuid5 y SHA-1.

El registro con PUT comprueba que no falten datos (código 400), evita cuentas duplicadas (código 409) y responde con un código 201 entregando el identificador y el token. El login con POST comprueba las credenciales frente al hash almacenado y devuelve un código 200 con el par de credenciales, o un código 401 si los datos son incorrectos. Por su parte, la actualización de contraseñas con PATCH exige enviar el token en la cabecera HTTP Authorization en formato Bearer, para actualizar la clave en memoria y confirmar la operación con un código 200.

Como se aprecia en la imagen 1, las pruebas con curl verifican la creación correcta del usuario (código 201), el rechazo al intentar registrarlo de nuevo (código 409) y el inicio de sesión válido (código 200). Asimismo, la primera parte de las pruebas automatizadas de la imagen 2 valida que el servicio responde adecuadamente ante todos los casos nominales y gestiona los errores de credenciales o campos ausentes.

```
marcos@marcos-HP-Laptop-15-fc0xxx:~/Documentos/3/SI/SI-1-main$ # Registro
curl -i -X PUT http://127.0.0.1:5050/user -H "Content-Type: application/json" -d '{"name":"juan","password":"123"}'

# Duplicado (Error 409)
curl -i -X PUT http://127.0.0.1:5050/user -H "Content-Type: application/json" -d '{"name":"juan","password":"123"}'

# Login correcto (200)
curl -i -X POST http://127.0.0.1:5050/user -H "Content-Type: application/json" -d '{"name":"juan","password":"123"}'
HTTP/1.1 201
content-type: application/json
content-length: 94
date: Fri, 02 Oct 2026 16:23:59 GMT
server: hypercorn-h11

{"token":"251fe6c7-e20a-55ad-bbff-279ef2a1305a","uid":"87a50156-359e-40c4-a5a4-56c385599ca1"}
HTTP/1.1 409
content-type: application/json
content-length: 32
date: Fri, 02 Oct 2026 16:23:59 GMT
server: hypercorn-h11

{"error":"user already exists"}
HTTP/1.1 200
content-type: application/json
content-length: 94
date: Fri, 02 Oct 2026 16:23:59 GMT
server: hypercorn-h11

{"token":"251fe6c7-e20a-55ad-bbff-279ef2a1305a","uid":"87a50156-359e-40c4-a5a4-56c385599ca1"}
```

Imagen 1

```
marcos@marcos-HP-Laptop-15-fc0xxx:~/Documentos/3/SI/SI-1-main$ python3 cliente.py

Pruebas de user.py

Probamos a crear un usuario válido
[OK] Código 201 al crear usuario
[OK] Devuelve UID y Token
[OK] Código 409 al intentar duplicar usuario
[OK] Código 400 por falta de contraseña

Probamos los inicios de sesión
[OK] Código 200 al iniciar sesión
[OK] El token devuelto es correcto
[OK] Código 401 con credenciales erróneas

Probamos el cambio de contraseña
[OK] Código 200 al actualizar contraseña
[OK] Código 200 al iniciar con la nueva contraseña
[OK] El token se mantiene igual tras cambio
[OK] Código 401 al intentar cambiar sin token
[OK] Código 400 en login sin credenciales
[OK] Código 400 en cambio de contraseña sin body
[OK] Código 401 en cambio de contraseña con token falso
```

Imagen 2

• **SERVICIO DE ARCHIVOS:**

El microservicio de archivos atiende en el puerto 5051 bajo rutas del tipo /file//... y se encarga del almacenamiento, lectura, listado, borrado y visibilidad de los documentos de cada usuario.

La autenticación en este componente es totalmente desacoplada y sin estado (*stateless*). Al compartir la misma clave secreta con el servicio de usuarios, no necesita realizar consultas por red ni conectarse a bases de datos externas para validar permisos. Cuando recibe una petición protegida con la cabecera Authorization: Bearer, calcula localmente el hash mediante uuid5 sobre el UID de la ruta y comprueba si coincide con el token recibido. Si encajan, autoriza la acción de forma directa.

El almacenamiento se organiza en disco dentro de la carpeta storage, creando un subdirectorio exclusivo para cada usuario identificado por su UID. La visibilidad se controla mediante un archivo auxiliar de metadatos public.json en cada carpeta, donde se indica si cada documento es público o privado. Las operaciones de subida con PUT, listado con GET, eliminación con DELETE y cambio de visibilidad con PATCH exigen siempre la autenticación del propietario devolviendo 401 si falta el token. Por su parte, la descarga de documentos mediante GET permite el acceso sin credenciales con código 200 si el archivo es público, pero responde con 401 si es privado y no se aporta el token legítimo.

En la imagen 3 se muestra la verificación con curl de esta política de acceso: tras subir un documento privado y uno público con código 201, el intento de lectura anónima sobre el privado devuelve un 401, mientras que el público se recupera sin credenciales con código 200 mostrando su contenido íntegro. Por otro lado, la segunda parte de las pruebas automatizadas de las imágenes 4 y 5 validan la respuesta ante archivos inexistentes con código 404, cuerpos incompletos con código 400 y el bloqueo estricto en el test multiusuario, impidiendo que un usuario acceda o borre datos de otro.

```
marcos@marcos-MP-Laptop-15-FC8xxx: /Documents/3/31/31-1-main$ # 1. Subir archivo privado
curl -i -X PUT http://127.0.0.1:5051/file/87a50156-359e-40c4-a5a4-56c385599ca1/secreto.txt -H "Content-Type: application/json" -H "Authorization: Bearer 251fe6c7-e20a-55ad-bbff-279ef2a1395a" -d '{"content": "Confidencial", "public": false}'

# 2. Intentar leerlo de forma anónima (debe devolver 401 Unauthorized)
curl -i -X GET http://127.0.0.1:5051/file/87a50156-359e-40c4-a5a4-56c385599ca1/secreto.txt

# 3. Subir archivo público
curl -i -X PUT http://127.0.0.1:5051/file/87a50156-359e-40c4-a5a4-56c385599ca1/nota.txt -H "Content-Type: application/json" -H "Authorization: Bearer 251fe6c7-e20a-55ad-bbff-279ef2a1395a" -d '{"content": "Publico", "public": true}'

# 4. Leer archivo publico de forma anónima sin token (debe devolver 200 OK)
curl -i -X GET http://127.0.0.1:5051/file/87a50156-359e-40c4-a5a4-56c385599ca1/nota.txt
HTTP/1.1 201
content-type: application/json
content-length: 38
date: Fri, 02 Oct 2026 16:32:31 GMT
server: hypercorn-h11

{"message": "file saved successfully"}
HTTP/1.1 401
content-type: application/json
content-length: 42
date: Fri, 02 Oct 2026 16:32:31 GMT
server: hypercorn-h11

{"error": "unauthorized. file is private"}
HTTP/1.1 201
content-type: application/json
content-length: 38
date: Fri, 02 Oct 2026 16:32:31 GMT
server: hypercorn-h11

{"message": "file saved successfully"}
HTTP/1.1 200
content-type: application/json
content-length: 44
date: Fri, 02 Oct 2026 16:32:31 GMT
server: hypercorn-h11

{"content": "Publico", "filename": "nota.txt"}
```

Imagen 3

```
Pruebas de file.py

Probamos a crear ficheros
[OK] Código 201 al crear archivo privado
[OK] Código 201 al crear archivo público
[OK] Código 401 al crear sin token

Probamos a listar el directorio
[OK] Código 200 al listar directorio
[OK] Aparecen todos los ficheros listados
[OK] Código 401 al intentar listar sin token
```

Imagen 4

```
-Probamos a leer ficheros
[OK] Código 200 al leer archivo privado propio
[OK] Código 401 al intentar leer archivo privado ajeno sin token
[OK] Código 200 al leer archivo público ajeno sin token
[OK] Contenido íntegro del archivo público

Probamos a modificar si el fichero es público
[OK] Código 200 al cambiar de privado a público
[OK] Código 200 al leer archivo recién expuesto al público

Probamos a borrar un archivo
[OK] Código 401 al borrar sin token
[OK] Código 200 al borrar archivo con token
[OK] Código 404 tras verificar que el archivo fue eliminado
[OK] Código 400 al crear archivo sin contenido
[OK] Código 404 al leer archivo inexistente
[OK] Código 401 al cambiar visibilidad sin token
[OK] Código 404 al cambiar visibilidad de archivo inexistente
[OK] Código 400 al cambiar visibilidad sin campo 'public'
[OK] Código 404 al borrar archivo que no existe
```

Imagen 5

- **GESTIÓN DE DOCKER:**

Para garantizar la portabilidad y aislar las dependencias de la aplicación sin alterar el sistema operativo base, hemos empaquetado ambos microservicios mediante contenedores Docker. La configuración del entorno parte de un archivo Dockerfile que utiliza la imagen de Python 3.12, establece el directorio de trabajo e instala las librerías de Quart y Requests indicadas en requirements.txt.

El arranque simultáneo de los servicios se gestiona mediante el archivo docker-compose.yml. En él se declaran dos servicios independientes, user_api y file_api, mapeando hacia la máquina física los puertos 5050 y 5051 respectivamente. Asimismo, se define un volumen montado sobre el directorio raíz para permitir que el almacenamiento persista fuera de los propios contenedores.

Como se aprecia en las imágenes 6 y 7, la ejecución del comando docker compose up compila las imágenes y levanta los contenedores, mientras que la salida de docker ps confirma que ambos se encuentran activos, ejecutando sus programas correspondientes y con los puertos enlazados de forma adecuada.

Por su parte, la persistencia en disco se valida en la imagen 8 y el cierre de los servicios en la imagen 9. En la primera se comprueba que la carpeta storage y el archivo public.json se han creado físicamente en el host con los permisos del contenedor, sobreviviendo al apagado del servicio. Por último, en la segunda captura se muestra la detención ordenada del entorno y la eliminación de los contenedores y de la red virtual mediante docker compose down.

```

marco@marcos-HP-Laptop-15-fdx8xx:~/Documents$ /usr/bin/docker compose up --build
[MARN] [0000] /home/marcos/Documents/3/SI/5I-1-main/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[MARN] [0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[*] Building 1.0s (16/19)

docker:default
-> [user_apl Internal] load build definition from Dockerfile
-> [user_apl Internal] load build definition from Dockerfile
-> [file_apl Internal] load build definition from Dockerfile
-> [user_apl Internal] load metadata for docker.io/library/python:3.12
-> [file_apl Internal] load .dockerignore
-> [user_apl Internal] load .dockerignore
-> [user_apl Internal] load build context
-> [user_apl Internal] load build context
-> [user_apl Internal] load build context
-> [user_apl 1/5] FROM docker.io/library/python:3.12@sha256:4f8bf79248325e6137f9d2dc7b6aff90578356fa2227f1b853d26ef9ab47
-> [file_apl Internal] load build context
-> [user_apl 2/5] WORKDIR /app
-> [user_apl 3/5] COPY requirements.txt
-> [user_apl 4/5] RUN pip install --no-cache-dir -r requirements.txt
-> [user_apl 5/5] COPY .

```

Imagen 6

```

== [user_api] exporting to image
== => exporting layers
== == exporting manifest sha256:66c208703c0961cc326fe77d0f8bdc8a553b4c7c3ba9e9cc9fd01e83ae3da
== == exporting config sha256:6a356d9802d0ebas1679733fad8914eac885fd0ea79ac00d180ccwad7e3bd
== == exporting attestation manifest sha256:e997b661771da70ff18923587e2a05f053a8d30af4f73b6c03a653309a7d1dc8
== == exporting manifest list sha256:a0c93f7610f9f5456b380c35e0eb998ba2304f544e7b3e5272bd0190e7b486631b6
== == naming to docker.io/library/si-1-main-user_api:latest
== == unpacking to docker.io/library/si-1-main-user_api:latest
== [file_api] exporting to image
== => exporting layers
== == exporting manifest sha256:b8ee31684587b704d97cac917363f1ceb7d7400d878e59394026af3917d8b48
== == exporting config sha256:12a3c2e72545c2529eed18d9d48137c313508486237bc54c857466e949c4005
== == exporting attestation manifest sha256:6e4953e0d31309787099599206c9c58601c54e1cded066343c29f08
== == exporting manifest list sha256:80e86ff0274-c26a87057a4889772796f405d3087e5504d208f5ae71ba78cafe
== == naming to docker.io/library/si-1-main-file_api:latest
== == unpacking to docker.io/library/si-1-main-file_api:latest
== [user_api] resolving provenance for metadata file
== [file_api] resolving provenance for metadata file
[+] Running 5/5
 ✓ file_api          Built
 ✓ user_api          Built
 ✓ Network si-1-main_default Created
 ✓ Container si-1-main-file_api-1 Created
 ✓ Container si-1-main-user_api-1 Created
Attaching to file_api-1, user_api-1

```

Imagen 7

```
marcos@marcos-HP-Laptop-15-fc0xxx:~/Documentos/3/SI/SI-1-main$ ls -la storage/
ls -la storage/*
cat storage/*/public.json
total 12
drwxr-xr-x 3 root root 4096 oct 2 18:32 .
drwxrwxr-x 3 marcos marcos 4096 oct 2 18:17 ..
drwxr-xr-x 2 root root 4096 oct 2 18:32 87a50156-359e-40c4-a5a4-56c385599ca1
total 20
drwxr-xr-x 2 root root 4096 oct 2 18:32 .
drwxr-xr-x 3 root root 4096 oct 2 18:32 ..
-rw-r--r-- 1 root root 7 oct 2 18:32 nota.txt
-rw-r--r-- 1 root root 64 oct 2 18:32 public.json
-rw-r--r-- 1 root root 12 oct 2 18:32 secreto.txt
{"secreto.txt": {"public": false}, "nota.txt": {"public": true}}marcos@marcos-HP-Laptop-15-fc0xxx:~/Documentos/3/SI/SI-1-main$
```

Imagen 8


```

marcos@marcos-HP-Laptop-15-fc0xxx:~/Documentos/3/SI/SI-1-main$ python3 cliente.py

Pruebas de user.py

Probamos a crear un usuario válido
[OK] Código 201 al crear usuario
[OK] Devuelve UID y Token
[OK] Código 409 al intentar duplicar usuario
[OK] Código 400 por falta de contraseña

Probamos los inicios de sesión
[OK] Código 200 al iniciar sesión
[OK] El token devuelto es correcto
[OK] Código 401 con credenciales erróneas

Probamos el cambio de contraseña
[OK] Código 200 al actualizar contraseña
[OK] Código 200 al iniciar con la nueva contraseña
[OK] El token se mantiene igual tras cambio
[OK] Código 401 al intentar cambiar sin token
[OK] Código 400 en login sin credenciales
[OK] Código 400 en cambio de contraseña sin body
[OK] Código 401 en cambio de contraseña con token falso

Pruebas de file.py

Probamos a crear ficheros
[OK] Código 201 al crear archivo privado
[OK] Código 201 al crear archivo público
[OK] Código 401 al crear sin token

Probamos a listar el directorio
[OK] Código 200 al listar directorio
[OK] Aparecen todos los ficheros listados
[OK] Código 401 al intentar listar sin token

```

Imagen 10

```

-Probamos a leer ficheros
[OK] Código 200 al leer archivo privado propio
[OK] Código 401 al intentar leer archivo privado ajeno sin token
[OK] Código 200 al leer archivo público ajeno sin token
[OK] Contenido íntegro del archivo público

Probamos a modificar si el fichero es público
[OK] Código 200 al cambiar de privado a público
[OK] Código 200 al leer archivo recién expuesto al público

Probamos a borrar un archivo
[OK] Código 401 al borrar sin token
[OK] Código 200 al borrar archivo con token
[OK] Código 404 tras verificar que el archivo fue eliminado
[OK] Código 400 al crear archivo sin contenido
[OK] Código 404 al leer archivo inexistente
[OK] Código 401 al cambiar visibilidad sin token
[OK] Código 404 al cambiar visibilidad de archivo inexistente
[OK] Código 400 al cambiar visibilidad sin campo 'public'
[OK] Código 404 al borrar archivo que no existe

Pruebas de seguridad multiusuario
[OK] Código 201 al crear segundo usuario (francisco)
[OK] Código 401: Francisco no puede leer archivo privado de Juan
[OK] Código 401: Francisco no puede listar los archivos de Juan
[OK] Código 401: Francisco no puede borrar archivos de Juan

PRUEBAS SUPERADAS: 39

PRUEBAS FALLIDAS: 0

```

Imagen 11